RISK ASSESSMENT · NEXT CHAPTER

thunk — Software Project Risk Assessment

Project / System: thunk (offline systems course for the reentry population)

Assessor: Penn Porterfield · **Date:** 2026

D / V / F = the dimension the risk threatens: `D` Desirability (do learners want / stay with it) `V` Viability (can it deploy and sustain) `F` Feasibility (can it be built). **L** = likelihood, **C** = consequence, each 1–5. **Risk = L × C** (max 25): `15–25 high` `8–14 medium` `1–7 low` .

RISK	HOW COULD IT HURT?	DVF	INITIAL	EXISTING CONTROLS	ADDITIONAL MITIGATION	FINAL
Curriculum scope unclear or underestimated	Scope creep, the Aug 2026 demo slips, burnout	D	16 4 × 4	Phased plan with documented non-goals; Phase 1 already built and passing tests	Freeze the demo to Phase 1; hold milestones; defer the web GUI, DOOM sim, and M6	6 2 × 3
Facility review blocks the software inside	The primary channel (inside the wall) closes; can't reach the core audience	V	15 3 × 5	Air-gapped single static binary; no network, no crypto; vocab-lint gate; runs on the walled-garden laptops facilities already deploy	Get a DOC/facility reviewer to look early; ship a plain-language review packet; pilot with one facility	8 2 × 4
Simulator can't convincingly reach the DOOM finale	The payoff and differentiator weakens; late rework	F	12 3 × 4	First sim slice built and tested (bus + panel + boot splash); SpiBus trait seam; doomgeneric is a known-portable target	Prototype DOOM-on-sim-panel as an early vertical slice; keep a simpler bootable demo as fallback	6 2 × 3
Insufficient testing or content errors	Wrong teaching, lost credibility, support burden	F	12 3 × 4	Unit tests across the workspace; self-validating check bank (tests assert every answer); fmt / clippy / vocab-lint in CI	Mentor / expert review of lessons; grow coverage; user-acceptance test with a pilot learner	6 2 × 3
Learners get discouraged and drop out	Weak completion and outcomes undercut the whole case	D	12 4 × 3	Self-paced and mastery-based; cannot get stuck; early visible payoff; progress is tracked	Small peer cohort plus a mentor; tiny early wins; measure completion in a pilot	6 2 × 3

RISK	HOW COULD IT HURT?	DVF	INITIAL	EXISTING CONTROLS	ADDITIONAL MITIGATION	FINAL
First-patch step overwhelms beginners; low-quality patches	Discouragement at the most important moment; reputational cost	D	9 3 × 3	Start small (docs, staging); plug into kernelnewbies / LKMP / Outreachy; meritocracy framing	Internal review before any patch is sent; a scaffolded first-issue list; a buddy for each first patch	4 2 × 2
No sponsor or funding to sustain / equip open-build learners	Hardware out of reach for outside learners; program longevity	V	9 3 × 3	Free and open source; simulator-first means a zero-hardware floor; the $200 Saleae writeup bounty is collectable now	Apply for grants; Saleae and partner outreach; keep the course fully usable with no hardware at all	4 2 × 2
Sole builder unavailable (key-person risk)	The project stalls; the lived-experience voice is lost	F	8 2 × 4	Specs, plans, and code are open source and documented (MIT / Apache-2.0)	Grow co-maintainers out of the community; keep a written knowledge base	6 2 × 3

Top residual risk: facility review (final 8, medium) is the one worth active attention, because it is mostly outside my control. Everything else drops into the low band once the mitigations are in place. The single most load-bearing control is the design itself: an offline, no-network, no-hardware static binary that was built to clear review from day one, which is why most of these risks were already low before mitigation.